Aalto University
School of Arts, Design
and Architecture

Check point no.7
Topic: Distributed Mixture-of-Experts (MoE) systems

Phuong Ngo Pitkänen and Amirreza Jafariandehkordi

When do we trigger rebalancing?

At each point, we collect:

- Tokens per expert

→ **Compute load per GPU**

→ **Identify the slowest GPU**

→ **Key metric:**

→ **Wasted fraction**

- Measures idle time caused by imbalance
- Based on difference between GPUs

→ **Decision rule:**

→ **If wasted fraction > threshold (1.5%) → trigger rebalancing**

How a New Mapping Is Computed

Mapping proposal

- First, estimate which experts are hot and which are cold
- Then we sort experts from heaviest to lightest
- We place each next heavy expert onto the currently least-loaded GPU

What this gives us

- A new **expert-to-GPU map**
- More even predicted GPU loads
- Lower expected wasted fraction

Important detail

It is not yet the full integrated with our cost model

Fallback Rebalancing Mechanism

Apply immediately during training	Start training with the current active map
	Count how many tokens are routed to each expert
	Estimate current GPU load and wasted fraction
	Check trigger condition
	Propose a better expert-to-GPU map
	If expected improvement is enough, update the active map
	Next steps use the new map immediately

Why live rebalancing is possible in fallback

- The fallback MoE is a custom lightweight path
- It directly reads expert_to_gpu_map during forward
- Remote experts are not truly moved across GPUs. We simulate this behavior manually
- So changing the map is mostly a planning and routing update

DeepSpeed Rebalancing Mechanism

For DeepSpeed, the workflow is not live:	Start training with a valid DeepSpeed expert layout
	Watch how heavily each expert is used
	Estimate imbalance using the same trigger logic
	Compute a better expert-to-GPU mapping
	Project that mapping into a DeepSpeed-compatible layout
	Save the proposed mapping to a JSON file
	Restart training with that saved map as the startup map

Why DeepSpeed Cannot Rebalance Live

To rebalance live in DeepSpeed, we would need to:

- move expert weights between GPUs
- update the router / gating
- update expert parallel groups
- update communication patterns
- update optimizer state
- update DeepSpeed internal structures

All that while training is going on. That would be quite complex.

After Loading the New Map

Metric	Step 3	Step 4
triggers	8	3
trigger rate	13.3%	5%
communication proxy (current)	1061	553
expected improvement	0.025	0.010

The startup map in Step 4 was already better than the default one
So the system started from a more balanced layout
As a result, fewer later corrections are needed

Next Steps

Current limitation

We have not yet fully integrated the full cost model into training

Right now we use a simplified version

Next step

Integrate the cost model directly into the planner

Timeline

Week 6-7
- Environment setup on CSC/LUMI
- Repository and shared documentation
- Literature review

Week 8-9
- Basic MoE training pipeline
- Multi-GPU experiments
- Initial benchmarks

Week 10-11
- Rebalancing techniques implementation

Week 12-13
- Experiments and evaluation

Week 14
- Final report writing and deliverables packaged

Week 15
- Demo and presentation

A!

Kiitos
aalto.fi